# 括号序列

## Part 1

考虑区间 DP，使用 $f[l][r]$ 表示区间 $[l,r]$ 构成合法序列的方案数， $g[l][r]$ 表示区间 $[l,r]$ 构成合法序列且两端括号配对的方案数。函数 `check` 则用于检查两端括号能否合法，预处理 $p[i]$ 表示由 $i$ 开始连续 `*` 的最大范围。

对于拼接规则，需要枚举划分的位置，时间复杂度为 $O(n^4)$，需要注意的是划分的位置要使第一部分两端括号配对，例如：

- `()()()` 只能划分为 `()` + `()()`，若划分为 `()()` + `()` 则产生重复
- `()()***()` 只能划分为 `()` + `()***()`，若划分为 `()()` + `***` + `()` 则产生重复

```
#include <bits/stdc++.h>
using namespace std;

const int N = 505, P = 1e9 + 7;
char s[N];
int n, k, f[N][N], g[N][N], p[N];

bool check(int l, int r) {
    return (s[l] == '(' || s[l] == '?') && (s[r] == ')' || s[r] == '?');
}
int search(int l) { //连续 * 的最大范围
    int r = l;
    while(s[r] == '*' || s[r] == '?') r++;
    return min(r, l + k);
}

int main() {
    scanf("%d%d%s", &n, &k, s);
    if(!check(0, n - 1)) return puts("0");

    for(int l = 0; l < n; l++) p[l] = search(l);
    for(int l = 0, r = 1; r < n; l++, r++) // 小区间初始化
        f[l][l] = g[l][l] = 0, f[l][r] = g[l][r] = check(l, r);

    for(int len = 2; len < n; len++) for(int l = 0, r = len; r < n; l++, r++) {
        if(!check(l, r)) continue; //无法配对

        long long tmp = f[l + 1][r - 1] + (p[l + 1] > r - 1); //(A),(S)

        for(int i = l + 1; i < r - 1; i++) {
            if(p[l + 1] > i) tmp += f[i + 1][r - 1]; //(SA)
            if(p[i + 1] > r - 1) tmp += f[l + 1][i]; //(AS)
        }
        g[l][r] = tmp %= P;

        for(int i = l + 1; i < r - 1; i++) {
            tmp += 1ll * g[l][i] * f[i + 1][r] % P; //AB
            for(int j = i + 2; j < r; j++)
                if(p[i + 1] >= j) tmp += 1ll * g[l][i] * f[j][r] % P; //ASB
        }
        f[l][r] = tmp % P;
    }

    printf("%d\n", f[0][n - 1]);
    return 0;
}
```

## Part 2

在 $ASB$ 这项规则的转移中，当枚举了第一个分界点 $i$ 后，第二个分界点 $j$ 产生的贡献是一个区间和，使用前缀和可以将时间复杂度优化为 $O(n^3)$。

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 505, P = 1e9 + 7;
char s[N];
int n, k, f[N][N], g[N][N], p[N], sum[N];

bool check(int l, int r) {
    return (s[l] == '(' || s[l] == '?') && (s[r] == ')' || s[r] == '?');
}
int search(int l) { //连续 * 的最大范围
    int r = l;
    while(s[r] == '*' || s[r] == '?') r++;
    return min(r, l + k);
}

int main() {
    scanf("%d%d%s", &n, &k, s);
    if(!check(0, n - 1)) return puts("0");

    for(int l = 0; l < n; l++) p[l] = search(l);
    for(int l = 0, r = 1; r < n; l++, r++) // 小区间初始化
        f[l][l] = g[l][l] = 0, f[l][r] = g[l][r] = check(l, r);

    for(int len = 2; len < n; len++) for(int l = 0, r = len; r < n; l++, r++) {
        if(!check(l, r)) continue; //无法配对

        long long tmp = f[l + 1][r - 1] + (p[l + 1] > r - 1); //(A),(S)

        for(int i = l + 1; i < r - 1; i++) {
            if(p[l + 1] > i) tmp += f[i + 1][r - 1]; //(SA)
            if(p[i + 1] > r - 1) tmp += f[l + 1][i]; //(AS)
        }
        g[l][r] = tmp %= P;

        for(int i = l + 1; i < r ; i++) //前缀和
            sum[i] = (sum[i - 1] + f[i][r]) % P;

        for(int i = l + 1; i < r - 1; i++) {
            tmp += 1ll * g[l][i] * f[i + 1][r] % P; //AB
            int j = min(r - 1, p[i + 1]);
            tmp += 1ll * g[l][i] * (sum[j] - sum[i + 1] + P) % P; //ASB
        }
        f[l][r] = tmp % P;
    }

    printf("%d\n", f[0][n - 1]);
    return 0;
}
```